Individual Project: Building and Evaluating a Basic Machine Learning Pipeline

Course: M505 Intro to AI and Machine Learning

In this individual project, students design, implement, evaluate, and report on a complete machine learning pipeline using Python and scikit-learn. The task covers the full workflow of a basic ML project: data collection, exploratory analysis, data cleaning and feature engineering, model training, hyperparameter tuning, visualization, interpretation, and conclusion.

Note: this document is for hints and recommendations. It does not replace the official M505 Assessment Brief.

1. Learning Objectives

After completing this assignment, students should be able to:

1. Select and download a suitable public dataset for a machine learning task.
2. Understand the structure, quality, and limitations of a dataset.
3. Apply basic data cleaning and preprocessing techniques using scikit-learn.
4. Build a reproducible machine learning pipeline.
5. Train and compare at least two different scikit-learn models.
6. Investigate the impact of hyperparameter settings on model performance.
7. Visualize data characteristics, model results, and evaluation outcomes.
8. Interpret results and write a clear technical report.
9. Export a Jupyter notebook as a well-structured HTML report.

2. Dataset Requirements

Students may choose any publicly available dataset from platforms such as Kaggle, HuggingFace Datasets, UCI Machine Learning Repository, OpenML, government open data portals, or other publicly accessible sources.

The dataset must satisfy the following requirements:

1. The dataset must be downloadable.
2. The dataset source must be clearly cited with a link.
3. The dataset must be suitable for a supervised machine learning task, such as classification or regression.
4. The dataset should contain enough samples and features to allow meaningful preprocessing and model comparison.
5. The dataset must not be too trivial. Extremely small toy datasets should be avoided unless the analysis is extended meaningfully.
6. The dataset must be manageable within a Jupyter notebook on a standard laptop.
7. Students must briefly explain why the dataset is suitable for this assignment.

3. Technical Constraints

Required tools:

- Python
- Jupyter Notebook
- scikit-learn
- pandas
- NumPy
- matplotlib or another basic visualization library

Allowed:

- pandas and NumPy for loading, inspecting, and basic manipulation of data.
- matplotlib for visualizations.
- scikit-learn for data preprocessing, feature transformation, feature selection, model training, pipelines, cross-validation, hyperparameter tuning, and evaluation metrics.

Not allowed:

- Deep learning frameworks such as TensorFlow, Keras, or PyTorch.
- Models outside scikit-learn.
- AutoML tools.
- Submissions containing only code without explanation. The notebook must read like a report.

4. Required Machine Learning Workflow

Stage 1: Problem Definition

Start the notebook with a clear problem statement. Students should answer:

- What is the prediction task?
- Is it a classification or regression problem?
- What is the target variable?
- What are the input features?
- Why is this problem meaningful?
- Who might benefit from solving this problem?

Example:

This project aims to predict whether a customer will churn based on demographic and service usage data. This is a binary classification problem because the target variable has two classes: churn and no churn.

Stage 2: Data Collection

Students should describe where the data comes from and include:

- Dataset name
- Dataset source
- Download link
- Number of samples
- Number of features
- Target variable
- Short description of the dataset

- License or usage information, if available

Students should show how the data is loaded into the notebook and explain whether the dataset was downloaded manually or programmatically.

```
import pandas as pd


df = pd.read_csv("dataset.csv")

df.head()
```

Stage 3: Exploratory Data Analysis

Students must perform an initial analysis of the dataset. The EDA should include:

- Dataset shape
- Feature names and data types
- Missing values
- Duplicate rows
- Target variable distribution
- Basic descriptive statistics
- Visualizations of important features
- Initial observations

For classification tasks, students should visualize the class distribution. For regression tasks, students should visualize the target variable distribution.

Example questions to answer:

- Are there missing values?
- Are there categorical variables?
- Are numerical features on different scales?
- Is the target variable balanced or imbalanced?
- Are there outliers?
- Are some features strongly correlated?

Stage 4: Data Cleaning and Feature Engineering

Students must apply data cleaning and feature engineering techniques supported by scikit-learn. They should explain the purpose and expected effect of each technique.

Missing Value Handling

- SimpleImputer
- Mean imputation
- Median imputation
- Most frequent value imputation
- Constant value imputation

```
from sklearn.impute import SimpleImputer


imputer = SimpleImputer(strategy="median")
```

Categorical Encoding

- OneHotEncoder
- OrdinalEncoder

```
from sklearn.preprocessing import OneHotEncoder
```

Feature Scaling

- StandardScaler
- MinMaxScaler
- RobustScaler

Students must illustrate the effect of at least one scaling technique, for example by comparing feature distributions before and after scaling, or by comparing model performance with and without scaling.

```
from sklearn.preprocessing import StandardScaler


scaler = StandardScaler()
```

Feature Transformation

- PolynomialFeatures
- KBinsDiscretizer
- FunctionTransformer

Feature Selection

- SelectKBest
- VarianceThreshold

Required Data Engineering Comparison

Students must compare at least two different preprocessing or feature engineering strategies. They should report whether the technique improved or reduced model performance and explain the result, not only show the score.

Strategy	Example Preprocessing
Option A	Missing value imputation + one-hot encoding
Option B	Missing value imputation + one-hot encoding + standard scaling
Option C	Missing value imputation + one-hot encoding + standard scaling + feature selection

5. Train-Test Split

Students must split the dataset into training and test sets using scikit-learn's train_test_split. For classification tasks, stratified splitting should be used when appropriate.

```
from sklearn.model_selection import train_test_split


X = df.drop("target", axis=1)
y = df["target"]


X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
```

```
    stratify=y
)
```

Students should explain why the dataset is split, what percentage is used for training and testing, why a random state is used, and whether stratification is necessary.

6. Machine Learning Models

Students must compare at least two different models supported by scikit-learn. The chosen models should be meaningfully different.

Task Type	Example Models
Classification	Logistic Regression; k-Nearest Neighbors; Decision Tree Classifier; Random Forest Classifier; Support Vector Machine; Naive Bayes; Gradient Boosting Classifier
Regression	Linear Regression; Ridge Regression; Lasso Regression; Decision Tree Regressor; Random Forest Regressor; Support Vector Regressor; Gradient Boosting Regressor

For each model, students should explain how the model works at a high level, why it is suitable for the chosen problem, and what assumptions or limitations it may have.

7. Training Pipeline

Students must use scikit-learn pipelines. The notebook should include a training pipeline that combines preprocessing and model training. Students are encouraged to use Pipeline, ColumnTransformer, GridSearchCV, RandomizedSearchCV, cross_val_score, or cross_validate.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features)
])
```

```
model_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", LogisticRegression())
])
```

Students should explain why using a pipeline is important: reproducibility, avoiding data leakage, cleaner code structure, and easier hyperparameter tuning.

8. Hyperparameter Tuning

Students must perform a deeper analysis of hyperparameter settings for at least two models. They should not only run default models. They must investigate how different hyperparameters affect performance.

Model	Example Hyperparameters	Interpretation
Logistic Regression	C; penalty; solver; max_iter	Smaller C means stronger regularization; larger C means weaker regularization.
Decision Tree	max_depth; min_samples_split; min_samples_leaf; criterion	Deeper trees may overfit; shallow trees may underfit.
Random Forest	n_estimators; max_depth; max_features; min_samples_split	More trees may improve stability but increase computation time; limiting depth can reduce overfitting.
k-Nearest Neighbors	n_neighbors; weights; metric	Small k may be sensitive to noise; large k may oversmooth predictions.
Support Vector Machine	C; kernel; gamma	C controls margin-error trade-off; gamma controls influence of training examples; kernel determines decision-boundary shape.

Required hyperparameter analysis:

1. Identify important hyperparameters.
2. Explain what each selected hyperparameter controls.
3. Test multiple values.
4. Report the results.
5. Discuss the effect of hyperparameter changes.
6. Select the best configuration based on validation or cross-validation results.

```
from sklearn.model_selection import GridSearchCV


param_grid = {
    "model__C": [0.01, 0.1, 1, 10, 100],
    "model__penalty": ["l2"],
    "model__solver": ["lbfgs"]
}
```

```
grid_search = GridSearchCV(
    model_pipeline,
    param_grid=param_grid,
    cv=5,
    scoring="accuracy"
)

grid_search.fit(X_train, y_train)

print(grid_search.best_params_)
print(grid_search.best_score_)
```

9. Model Evaluation

Students must evaluate the final models on the test set using metrics appropriate to the task.

For Classification Tasks

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- ROC-AUC, if appropriate

```
from sklearn.metrics import classification_report, confusion_matrix

y_pred = best_model.predict(X_test)

print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

Students should explain which metric is most important for the problem. For example, in medical diagnosis, recall may be more important than accuracy. In spam detection, precision may be important to avoid marking real emails as spam.

For Regression Tasks

- Mean Absolute Error
- Mean Squared Error
- Root Mean Squared Error
- R^2 score

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Students should explain what each metric means and which one is most relevant for the problem.

10. Visualization Requirements

The notebook must include meaningful visualizations. At minimum, it should include:

1. Target variable distribution.

2. At least two feature visualizations.

3. Visualization of missing values or data quality issues.

4. Comparison of model performance.

5. Visualization related to hyperparameter tuning.

6. Visualization of the final model results.

7. Visualization of the training process, e.g., loss function.

Students should not include visualizations without explanation. Each visualization must be followed by a short interpretation.

11. Report and Interpretation

The notebook must be written as a report, not just as code. Each major section should include markdown explanations. The report should answer:

- What dataset was used?
- What problem was solved?
- What preprocessing steps were necessary?
- Which data engineering techniques were tested?
- How did different preprocessing strategies affect performance?
- Which models were compared?
- Which hyperparameters were tuned?
- Which model performed best?
- Why did it perform best?
- What are the limitations of the analysis?
- What could be improved in future work?

12. Required Notebook Structure

Notebook Section	Required Content
1. Title and Student Information	Assignment title, student name, course name, and date.
2. Introduction	Problem statement, dataset overview, machine learning task, and motivation.
3. Data Collection	Dataset source, download information, loading code, and dataset description.
4. Exploratory Data Analysis	Data shape, feature types, missing values, target distribution, descriptive statistics, and initial visualizations.
5. Data Cleaning and Feature Engineering	Missing value handling, encoding, scaling, transformations, and explanation of preprocessing choices.
6. Data Engineering Comparison	At least two preprocessing strategies, performance comparison, and explanation of effects.
7. Model Training	Train-test split, at least two scikit-learn models, basic training results, and pipeline implementation.
8. Hyperparameter Tuning	Hyperparameter grids, cross-validation setup, best parameters, comparison of tuning results, and interpretation.

9. Final Evaluation	Test set performance, metrics, error analysis, and model comparison.
10. Visualization and Discussion	Performance plots, hyperparameter effect plots, confusion matrix, ROC curve, residual plot, or similar.
11. Conclusion	Summary of findings, best model, lessons learned, limitations, and future improvements.
12. References	Dataset source, scikit-learn documentation pages used, and any tutorials or external resources consulted.

13. Submission Format

Students must submit:

1. The exported HTML version of the notebook: .html

2. Keep the original Jupyter notebook file: .ipynb as backup.

The HTML file must be generated from the Jupyter notebook.

```
jupyter nbconvert --to html assignment.ipynb
```

The HTML report should include code, outputs, tables, visualizations, and markdown explanations. Before submission, students must make sure the notebook runs from top to bottom without errors.

14. Reproducibility Requirements

The notebook must be reproducible. Students should:

1. Use a fixed random_state where appropriate.

2. Clearly document how to obtain the dataset.

3. Avoid hard-coded local paths that only work on one computer.

4. Ensure all code cells run in order.

5. Include all necessary imports.

6. Make sure the final HTML contains all outputs.

7. Explain any manual steps required to download the data.

Good example:

```
RANDOM_STATE = 42
```

Poor example:

```
df = pd.read_csv("C:/Users/MyName/Desktop/random_folder/data.csv")
```

Better example:

```
df = pd.read_csv("data/dataset.csv")
```

15. Suggested Datasets

Classification examples:

- Titanic survival prediction
- Customer churn prediction

- Heart disease prediction
- Bank marketing classification
- Spam detection
- Credit risk classification
- Mushroom classification
- Student performance classification

Regression examples:

- House price prediction
- Car price prediction
- Energy consumption prediction
- Bike sharing demand prediction
- Student score prediction
- Medical insurance cost prediction
- Air quality prediction

16. Common Mistakes to Avoid

1. Training on the full dataset without a test split.
2. Performing preprocessing before splitting the data in a way that causes data leakage.
3. Using only one model.
4. Using only default hyperparameters.
5. Reporting scores without interpretation.
6. Choosing metrics that do not fit the task.
7. Using models outside scikit-learn.
8. Using deep learning models.
9. Submitting only code without markdown explanations.
10. Using visualizations without discussing them.
11. Using a dataset that cannot be downloaded or verified.
12. Submitting a notebook that does not run from start to finish.
13. Forgetting to export the notebook as HTML.
14. Comparing models unfairly with different preprocessing steps unless this is intentionally discussed.
15. Ignoring class imbalance in classification problems.